Section Overview

- Introduction to CUDA, SGEMM, and the Naive Kernel

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

Example

During the forward pass when training a model, we accumulate the gradients from the entire batch.

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

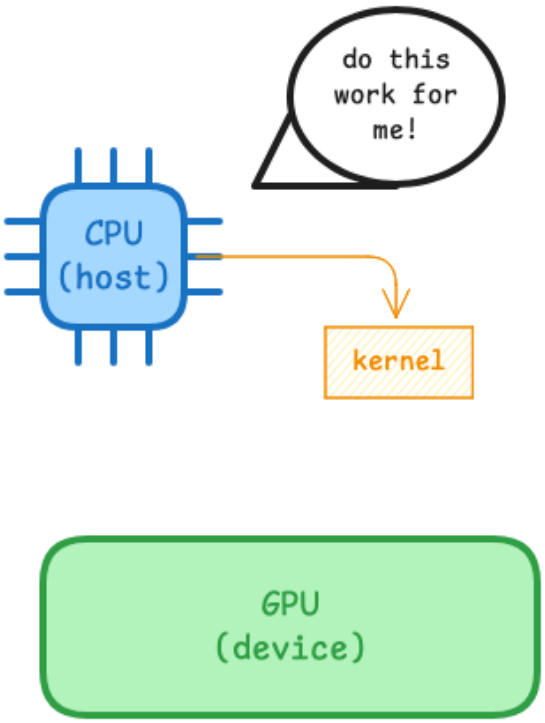
```
sgemm(
  int M, int N, int K,
  float alpha, float *A, float *B,
  float beta, float *C
)
```

$$\underbrace{A}_{M\times K} \quad \underbrace{B}_{K\times N} \quad \underbrace{C}_{M\times N}$$

- M: rows of A and C
- N: columns of B and C
- K: shared reduction dimension

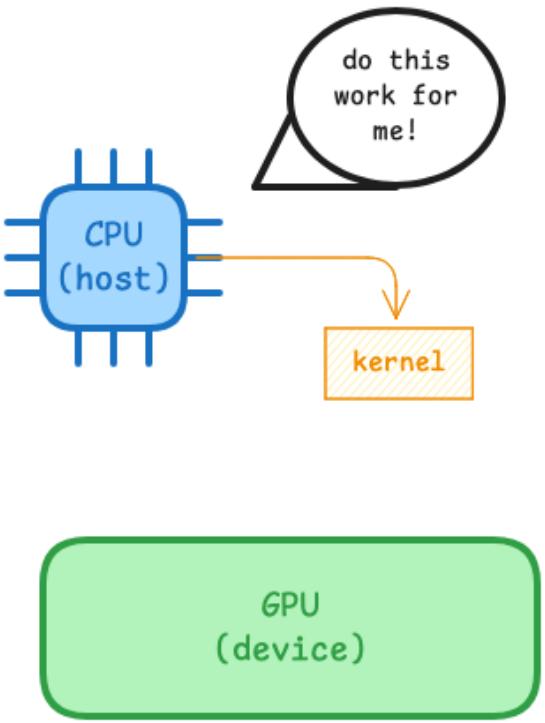
CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.



CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.




```
int main() {
  const int M = 128, N = 128, K = 128;

  float *dA, *dB, *dC;
  cudaMalloc(&dA, M * K * sizeof(float);
  cudaMalloc(&dB, K * N * sizeof(float));
  cudaMalloc(&dC, M * N * sizeof(float));

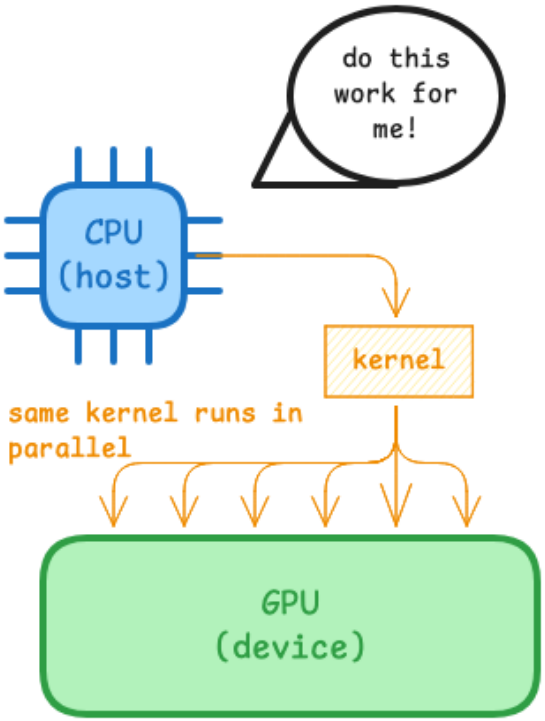
  // load values into these device pointers...

  run_sgemm_naive(M, N, K, 1, dA, dB, 0, dC);

  cudaFree(dA); cudaFree(dB); cudaFree(dC);
}
```

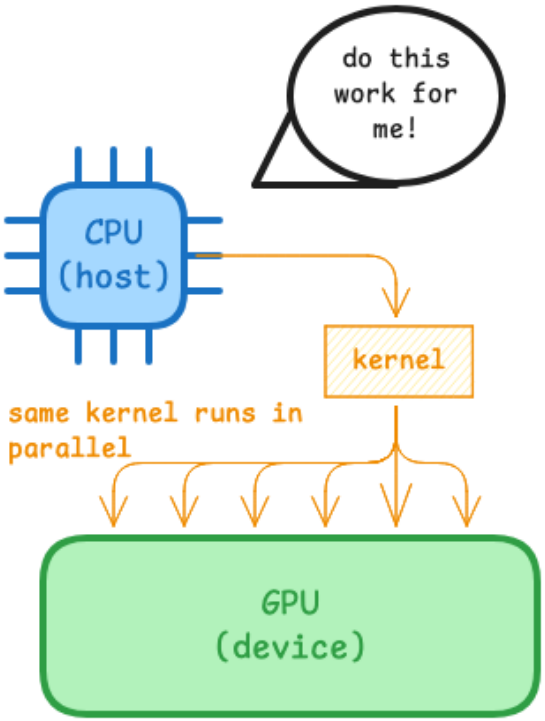
CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.



CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.

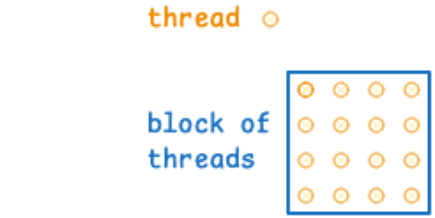



```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}
```

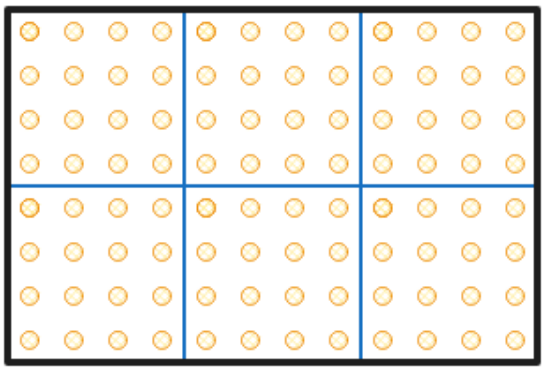
```
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

CUDA Basics (Threads, Blocks, Grids)

- Threads are grouped into **blocks**.
- The collection of blocks requested by the host is organized in a **grid**.



Grid of blocks



```cpp
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}
```

```cpp
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

CUDA Basics (Example)

Example

We want to do a plain matrix multiplication between matrices A and B and store it into C.

CUDA Basics (Example)

Example

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000.

CUDA Basics (Example)

Example

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000.

CUDA Basics (Example)

Example

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

CUDA Basics (Example)

Example

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.
We want blocks of 32×32, so $BM, BN = 32$.

```
int M = 5000, N = 4000, K = 3000;
float alpha = 1, beta = 0;

float *dA, *dB, *dC; // Allocate buffers on the DEVICE.
cudaMalloc(&dA, M * K * sizeof(float);
cudaMalloc(&dB, K * N * sizeof(float));
cudaMalloc(&dC, M * N * sizeof(float));
// BM, BN are compile-time constants!
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
```

CUDA Basics (Example)

Example

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
// inside launcher function
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C)
```

CUDA Basics (Example)

Example

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
// inside launcher function
  dim3 blockDim(32, 32);
  dim3 gridDim((4000 + 32 - 1) / 32, (5000 + 32 - 1) / 32); // Ceil division
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C)
```

CUDA Basics (Example)

Example

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
// inside launcher function
  dim3 blockDim(32, 32);
  dim3 gridDim(126, 158); // Ceil division
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C)
```

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Relation to hardware

This hierarchy is closely related to how the device schedules the execution. We will see later how to use this to optimize performance.

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Relation to hardware

This hierarchy is closely related to how the device schedules the execution.
We will see later how to use this to optimize performance.

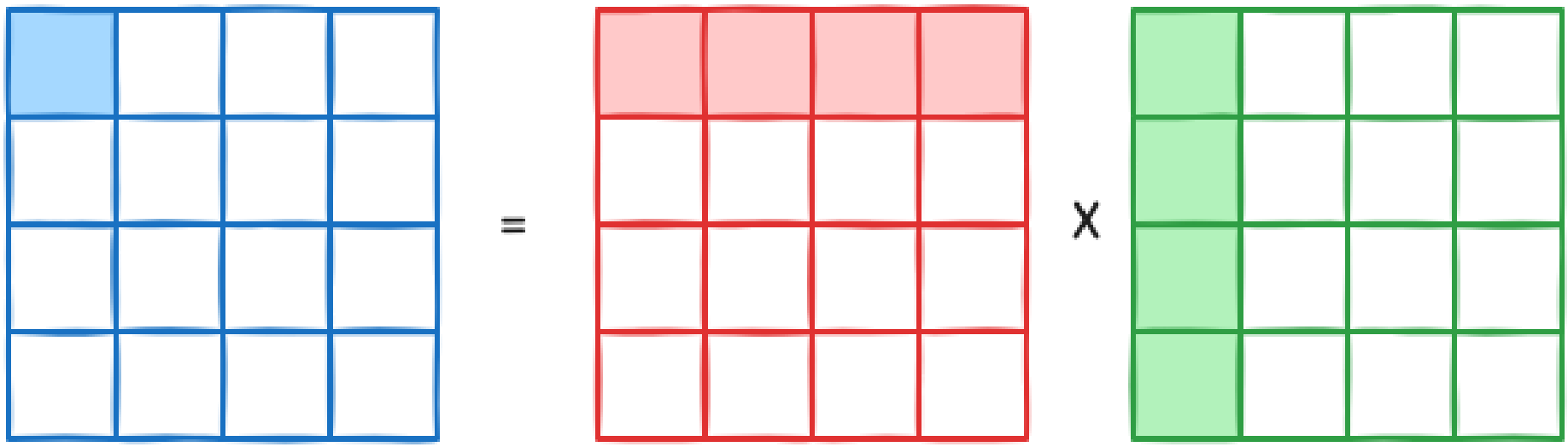
Thread, Block, Grid hierarchy

Threads are grouped into blocks of constant size/shape.
Blocks are launched as a grid.
We choose the dimensions of the grid to control how many blocks are launched, and thus how many threads are launched.

Let's design a SGEMM kernel!

Recall

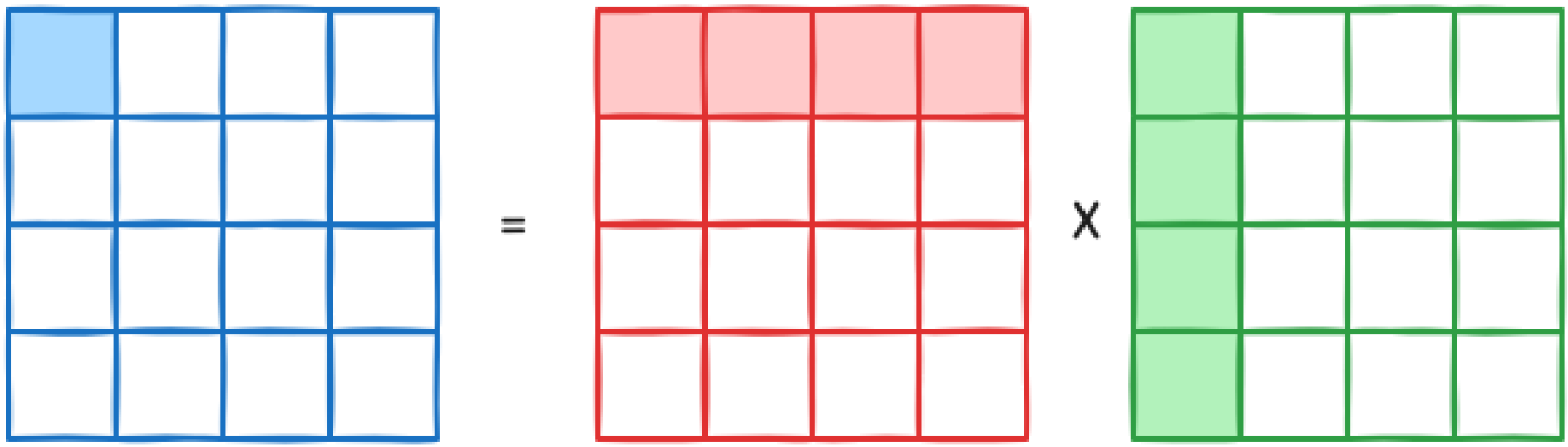
Recall that matrix multiplication is naturally parallel, and the output elements of C can be computed independently in parallel.



Let's design a SGEMM kernel!

Recall

Recall that matrix multiplication is naturally parallel, and the output elements of C can be computed independently in parallel.



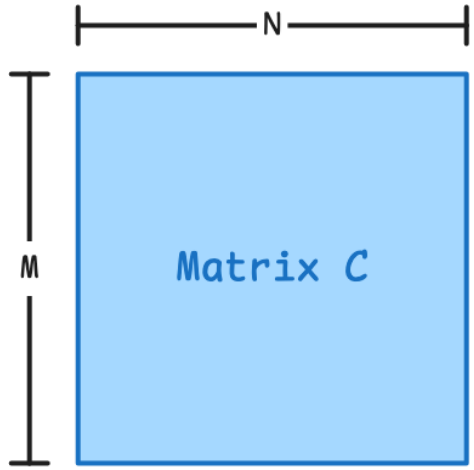
We can naturally map one thread to each output element!

Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

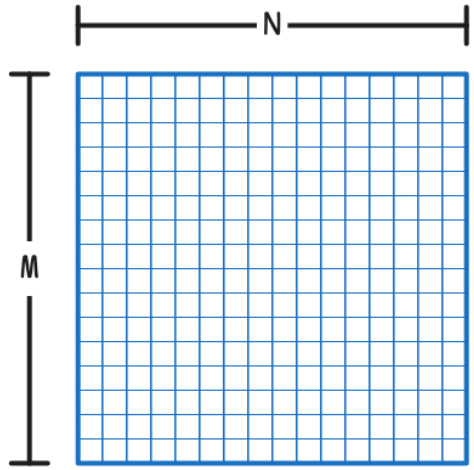


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

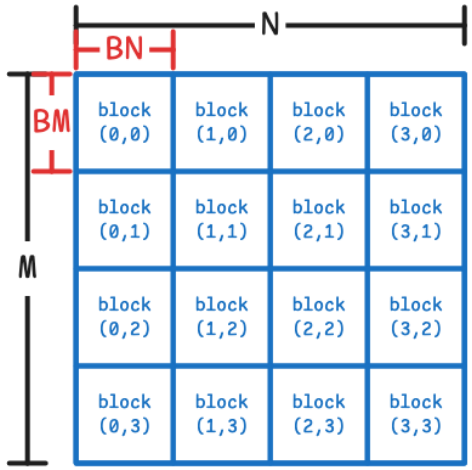


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

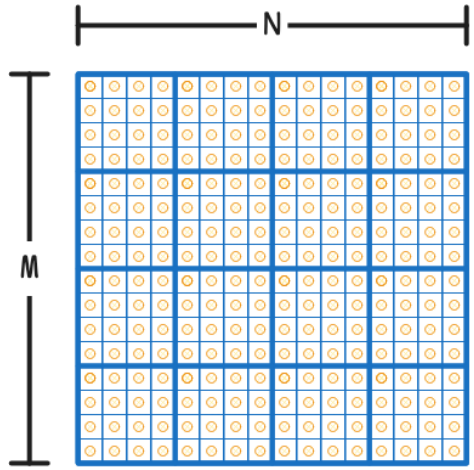


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

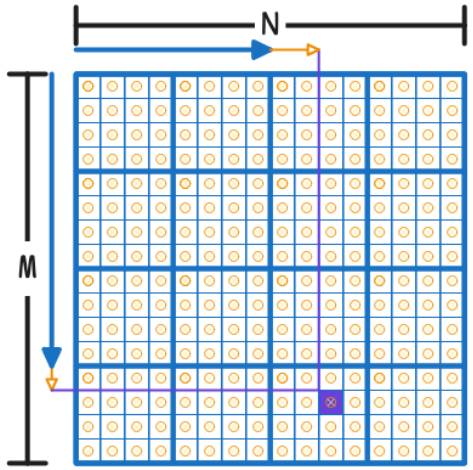


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```



Naive SGEMM visualization

- One thread computes one element of C.
- It walks one row of A and one column of B for the full dot product.

Play naive-kernel visualization

Naive CUDA is already faster than the CPU

